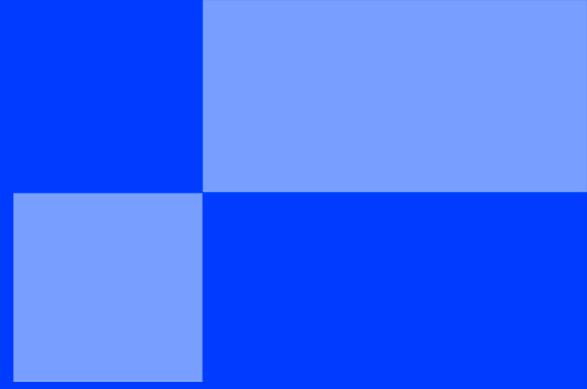




SUMMER SCHOOL

< PART 4 BOOTSTRAP - THE LOCAL TUNNEL
PIPELINE (NGROK) />

SUMMER SCHOOL

< AI & AUTOMATION: BRIDGING CLOUD AND LOCALHOST />

"Make it work, make it right, make it fast."
— **Kent Beck**

Introduction

In the previous modules, you played exclusively in the "Cloud". Make.com caught data from Google Forms and sent it to Gmail. But as a Full-Stack developer, your goal is to connect external tools directly to the code running on your own computer.

There is a major network problem: Make.com is a public server on the internet, but your Express Back-End runs on `http://localhost:3000`. Localhost is a private, locked room inside your computer. Make.com cannot see it.

To solve this, we will use **Ngrok**. Ngrok drills a secure tunnel through your router's firewall and gives your local server a temporary public URL.

In this Bootstrap, we will build a simple proof-of-concept: sending a custom message from Make.com directly into your local computer terminal.

Step 1: Preparing the Local Catcher (Express)

Before we open a tunnel, we need something listening on the other side. Let's create a tiny, disposable Express server whose only job is to print whatever Make.com sends it.

1. Open a new terminal and create a quick sandbox:

```
mkdir ngrok-sandbox
cd ngrok-sandbox
npm init -y
npm install express
```

2. Create a file named **server.js** and paste this code:

```
const express = require('express');
const app = express();

// This allows our server to read incoming JSON data
app.use(express.json());

// We create a specific webhook route to catch the data
app.post('/webhook', (req, res) => {
  console.log("BOOM! Data received from the Cloud:");
  console.log(req.body); // Print the Make.com payload to the terminal
  res.status(200).send("Message received loud and clear!");
});

app.listen(3000, () => {
  console.log('Sandbox server listening on port 3000');
});
```

3. Start your server: **node server.js**. Keep this terminal open!

Step 2: Opening the Tunnel (Ngrok)

Your server is ready, but it's invisible to the internet. Let's expose it.

1. Go to ngrok.com/download and create a free account.
2. Download the executable file for your specific operating system (Windows, macOS, or Linux) and unzip it into a known folder.
3. Find your **Authtoken** on your Ngrok dashboard. Open a **new, second terminal window** in the folder where you unzipped Ngrok, and run the configuration command to link your account:

```
# On Mac/Linux:
ngrok config add-authtoken YOUR_TOKEN_HERE

# On Windows:
ngrok.exe config add-authtoken YOUR_TOKEN_HERE
```

4. Start the tunnel to expose your local port 3000:

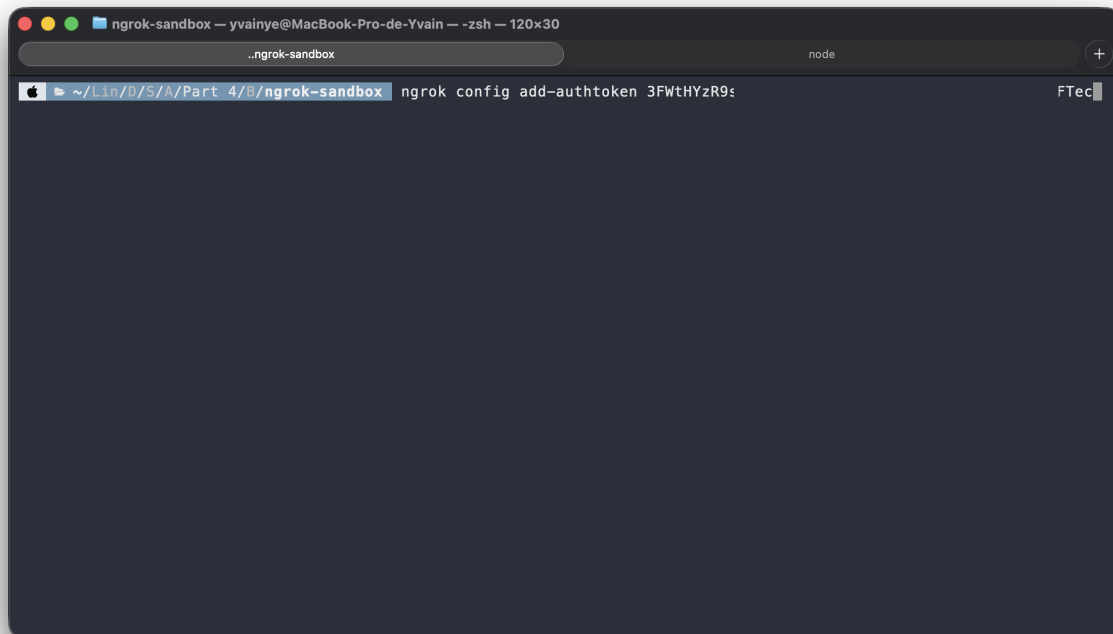
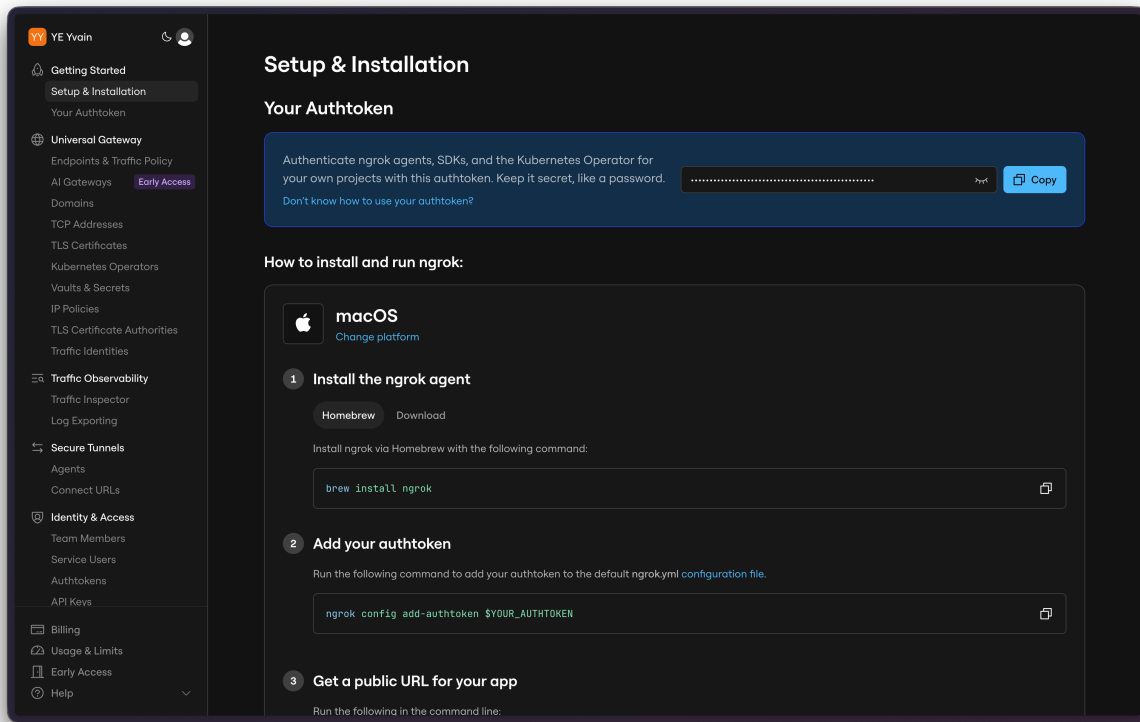
```
# On Mac/Linux:
ngrok http 3000

# On Windows:
ngrok.exe http 3000
```

5. A black screen will appear with connection details. Look for the **Forwarding** line.
6. Copy the secure URL (it looks something like **https://a1b2-34-56.ngrok-free.app**).
7. **DO NOT close this terminal!** If you close it, the tunnel collapses and the URL dies.



How it works: Anyone on the planet can now visit your ngrok-free.app link, and the request will instantly be forwarded to your hidden localhost on port 3000.



```
ngrok-sandbox — ngrok http 3000 — ngrok http 3000 — 120x30
ngrok
One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai
Session Status      online
Account             YE Yvain (Plan: Free)
Version             3.39.8
Region              Europe (eu)
Latency              20ms
Web Interface        http://127.0.0.1:4040
Forwarding           https://jawless-covenant-kitchen.ngrok-free.dev -> http://localhost:3000
Connections
  ttl   opn   rt1   rt5   p50   p90
    0    0    0.00  0.00  0.00  0.00
```

```
ngrok-sandbox — node server.js — node server.js — 120x30
node server.js
Sandbox server listening on port 3000
```

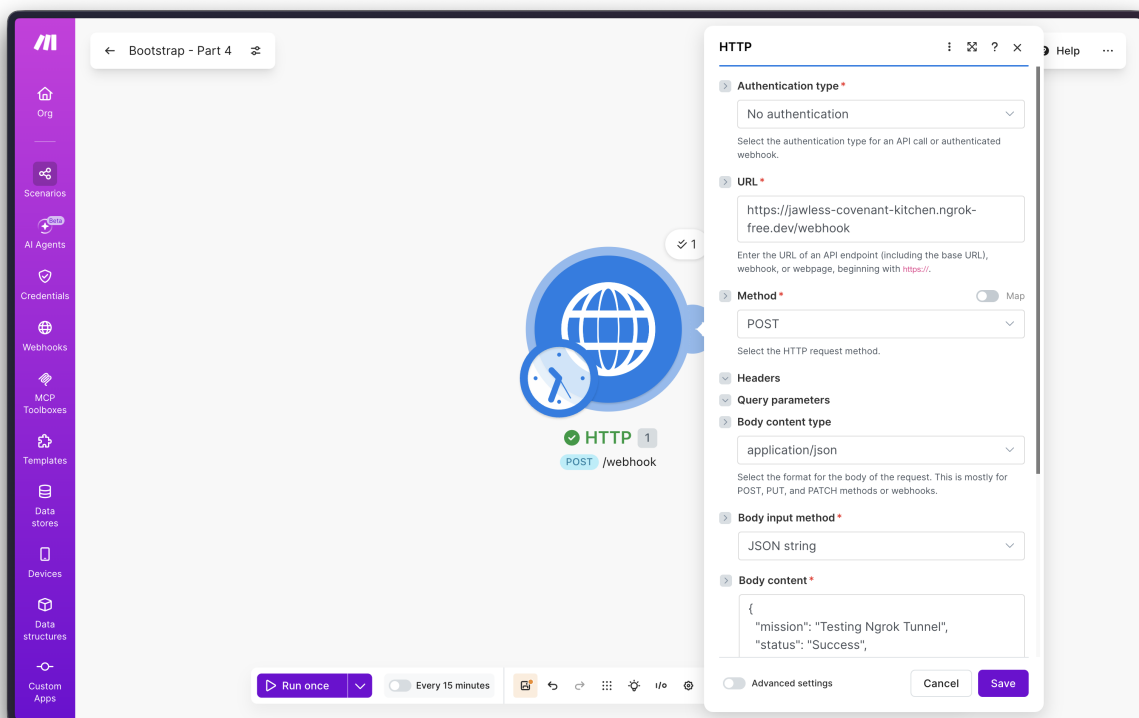
Step 3: The Make.com HTTP Module

Now that we have a public URL, let's configure our automation to shoot data at it.

1. Go to Make.com and create a new scenario.
2. For the **Trigger**, select a simple **Tools -> Basic trigger** (or use your previous Google Form).
3. Click to add a new module. Search for **HTTP** and select **Make a request**.
4. Fill in the HTTP settings exactly like this:
 - ✓ **URL:** Paste your Ngrok URL and add **/webhook** at the end (e.g., **https://xyz.ngrok-free.app/webhook**).
 - ✓ **Method:** POST
 - ✓ **Body type:** Raw
 - ✓ **Content type:** JSON (application/json)
 - ✓ **Request content:** Write a simple JSON object:

```
{  
  "mission": "Testing Ngrok Tunnel",  
  "status": "Success",  
  "agent": "Make.com"  
}
```

5. Click **Save**.



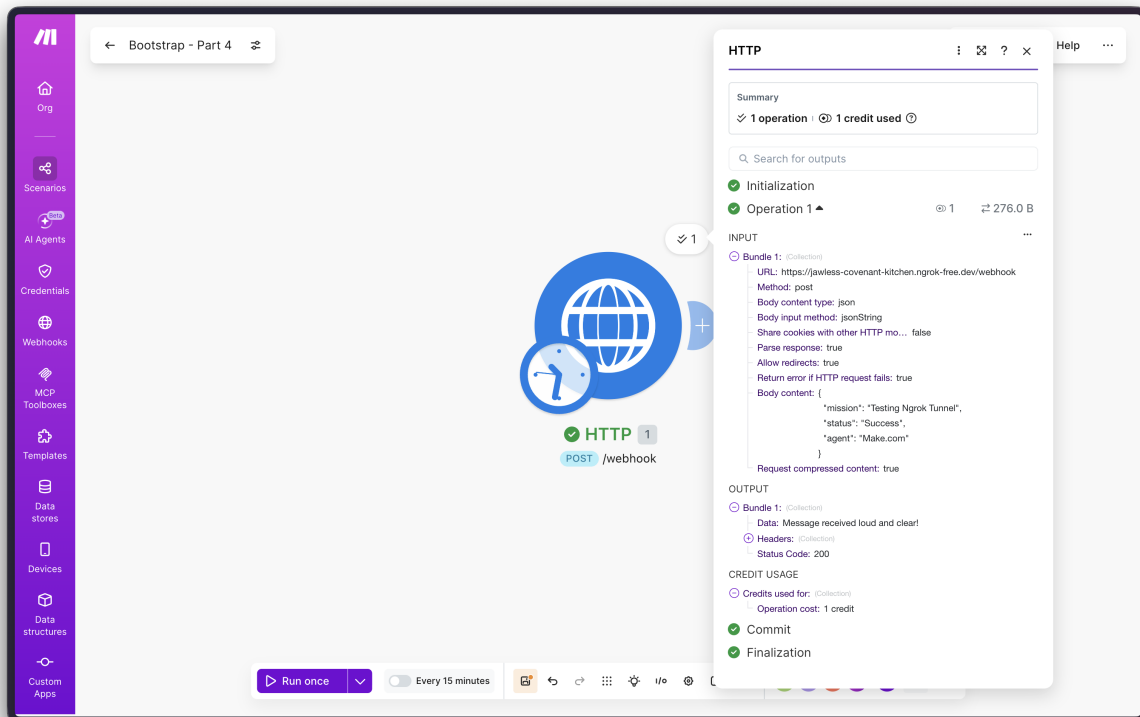
Step 4: The Big Test

It is time to fire the laser. Arrange your windows so you can see Make.com and your **Express server terminal** (the first one) at the same time.

1. On Make.com, click **Run once** at the bottom left.
2. The HTTP module will flash green, meaning the request was successfully fired into the internet.
3. Look at your local terminal! You should instantly see:

BOOM! Data received from the Cloud:

```
{ mission: 'Testing Ngrok Tunnel', status: 'Success', agent: 'Make.com' }
```



```
~/Lin/D/S/A/Part 4/B/ngrok-sandbox node server.js
Sandbox server listening on port 3000
BOOM! Data received from the Cloud:
{
  mission: 'Testing Ngrok Tunnel',
  status: 'Success',
  agent: 'Make.com'
}
```


Conclusion

Congratulations! You have successfully bridged the gap between a cloud automation platform and your local development environment.

You understand how to expose a local port using Ngrok and how to construct a custom HTTP POST request in Make.com.

You are now ready to tackle the **Project**, where you will replace this sandbox server with your real Newspaper Backend and inject real form data into your **articles.json** database!

v1

{EPITECH}